

CN7050 – Intelligent Systems

Week 7: Retrieval Augmented Generation (RAG) Systems

Dr Shaheen Khatoon

Office: EB.1.99

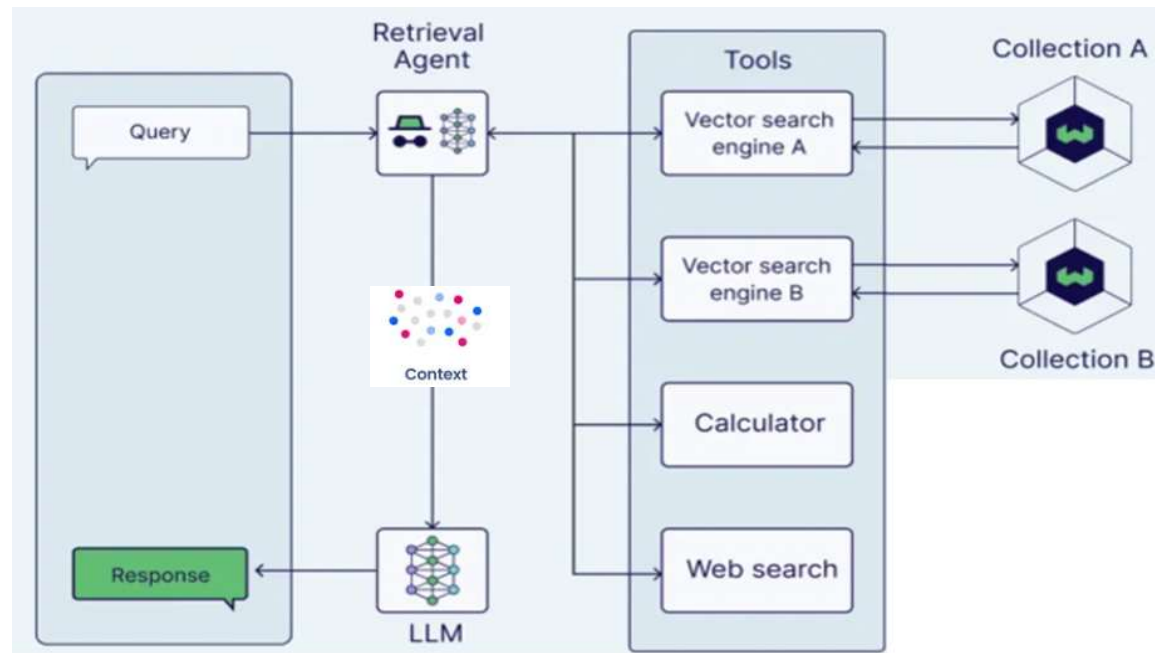
Email: s.khatoon@uel.ac.uk

RAG Motivations

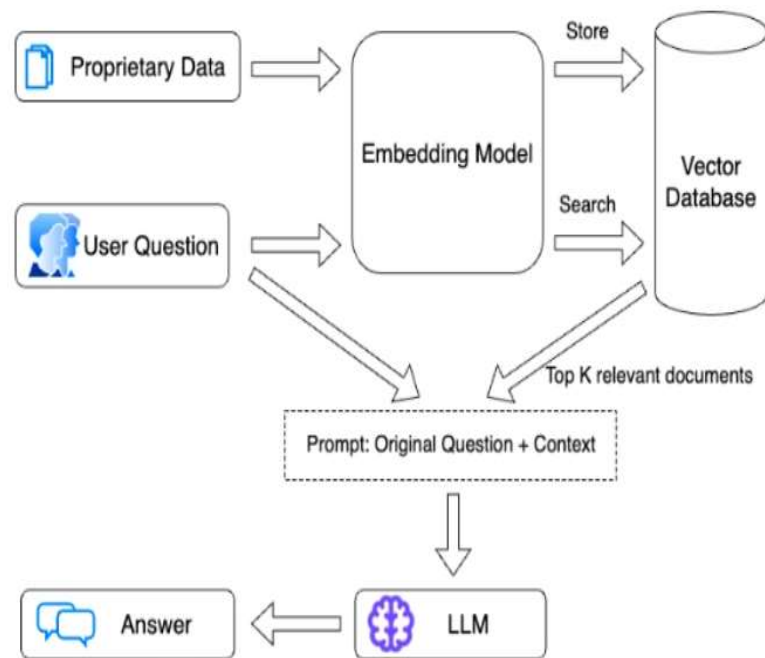
- Large Language Models operate on their internal, pre-trained knowledge.
- This approach has a clear limitation:
 - The model is unaware of any data created after its training cut-off date
 - Has no access to private or domain-specific information.
 - They are inefficient when dealing with specific knowledge
- Retrieval-Augmented Generation (RAG) overcome this by Integrating external, up-to-date information into their responses.
 - first retrieve relevant information from an external database/knowledge base of documents, and then
 - provide them to the LLM as context for generating a response.
- This grounds the model's output in factual, timely, and specific information.

Agentic RAG Architecture

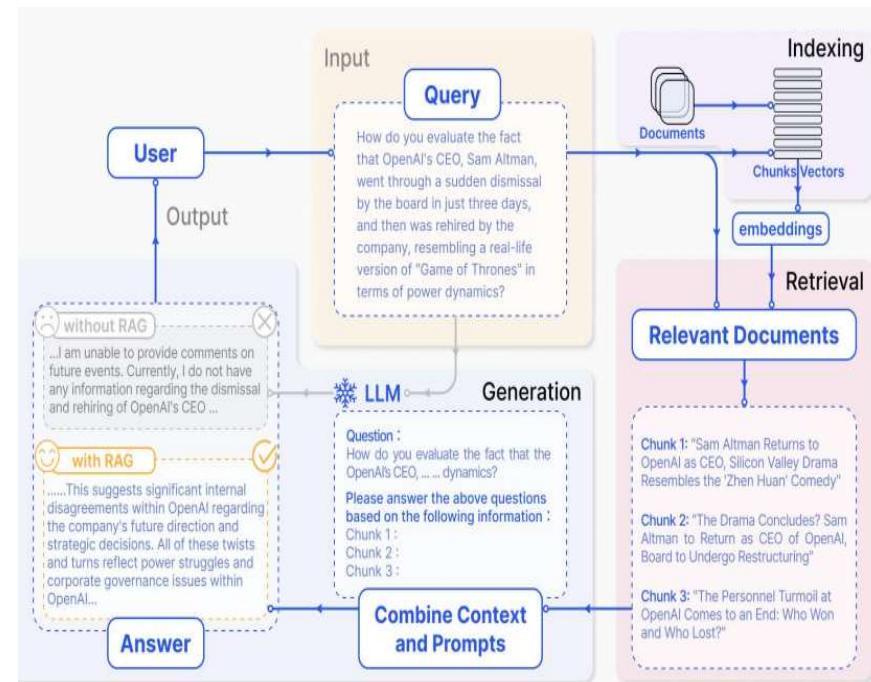
- Retrieval Agent: retrieve relevant data from external sources, such as databases or documents
- Generator component (LLM): Use retrieved data to generate more accurate and contextually appropriate answers



Traditional RAG Architecture



RAG Process



A typical case of RAG

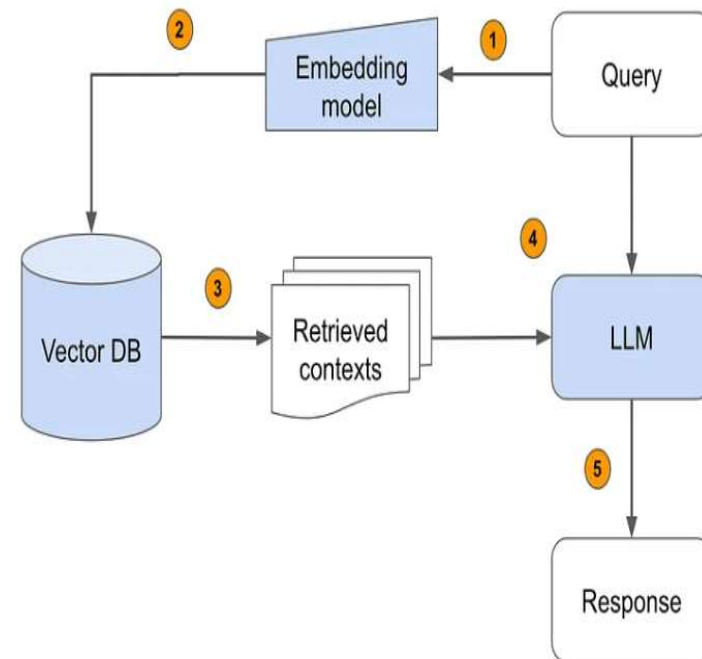
Understanding RAG Architecture

1. Data Processing and Indexing:

- **Data Preparation:** Collect and preprocess external data sources, which can be **unstructured text, semi-structured data, or structured knowledge graphs**.
- **Embedding Creation:** Convert the prepared data into **vector representations (embeddings)** that capture semantic meanings.
- **Storage:** Store these embeddings in a **vector database**, enabling efficient **similarity searches**

2. Retrieval Mechanism:

- **Query Encoding:** When a user submits a query, **encode it into a vector using the same embedding model** employed during **data indexing**.
- **Similarity Search:** Compare the **query vector** against the **stored embeddings** in the vector database to **identify and retrieve the most relevant documents or data chunks**.



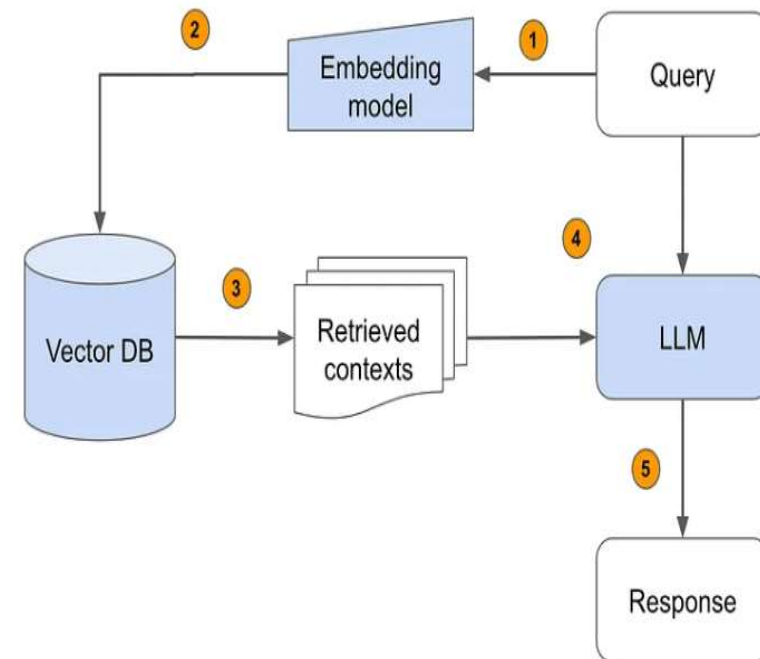
Understanding RAG Architecture

3. Augmentation Process:

- **Contextual Integration:** Combine the retrieved information with the original user query to form an **augmented input**.
- **Prompt Engineering:** Structure this augmented input into a format suitable for the LLM, ensuring that the model can effectively utilize the additional context.

3. Generation Phase:

- **Response Generation:** Input the **augmented prompt** into the LLM, which generates a response that incorporates both its internal knowledge and the retrieved external information.
- **Output Delivery:** Present the **generated response** to the user, often with citations or references to the external sources used, enhancing transparency and trust.



Exploring RAG Libraries

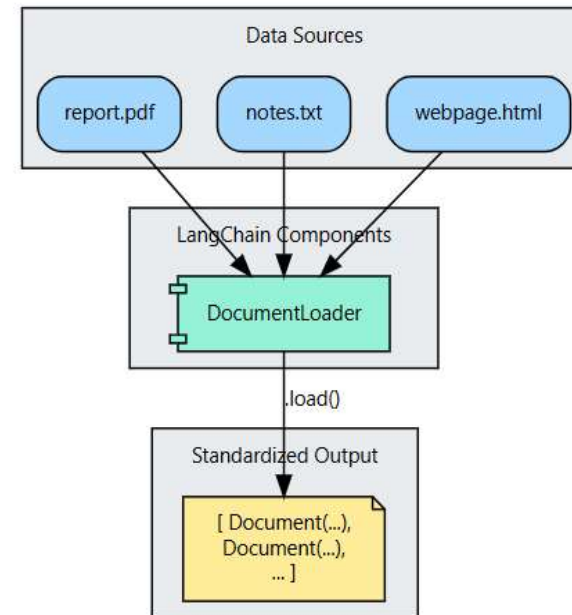
- Several different libraries that enable us to be able to build such a system:
 - **LangChain**: framework for developing applications with LLMs at their core.
 - **Haystack**: is an open source framework for building production-ready LLM applications.
 - **LlamaIndex**: is another framework focused on building a system around an LLM.
 - **Semantic Kernel**: is an open source framework developed by Microsoft to build agents.
 - **AutoGen**: is a Python-based LLM framework developed by Microsoft and other entities (University of Washington, Penn State, and Xidian). The idea behind it is to build applications that configure multiple agents that communicate with each other to complete a task.
- In general, the different frameworks offer similar components and are inspired by the same philosophy (extending the capabilities of an LLM).

Developing RAG using LangChain Framework

- Construct a RAG system piece by piece, learning how to:
 - **Load Documents:** Use LangChain's *DocumentLoaders* to ingest data from various file formats like **PDFs, Web, text files** etc.
 - **Process Text:** Split large documents into smaller, manageable **chunks** using **TextSplitters**.
 - **Create Embeddings:** Convert text chunks into **numerical vectors**, which allows for **semantic comparison**.
 - **Store and Retrieve:** Use a **VectorStore** to index these embeddings for efficient similarity searches, where relevance is determined by the distance between a query *vector vq* and document *vectors vd* .
 - **Build a Q&A Chain:** Combine these components with a **retriever and an LLM** to build a complete question-answering application that can reason over your own data.

Document Loading

- **Load document** which can exist in various formats like PDF files, web pages, text files, or entries in a database
- **DocumentLoader** is used to ingest this data into a standardized format that LangChain can work with.
- Each loaded source is represented as a **Document** object, which contains the *text content* and associated *metadata* (*additional information such as source, page number, or creation date.*)



Document Loading

■ Example of DocumentLoader

▪ Text Files

```
from langchain_community.document_loaders import TextLoader

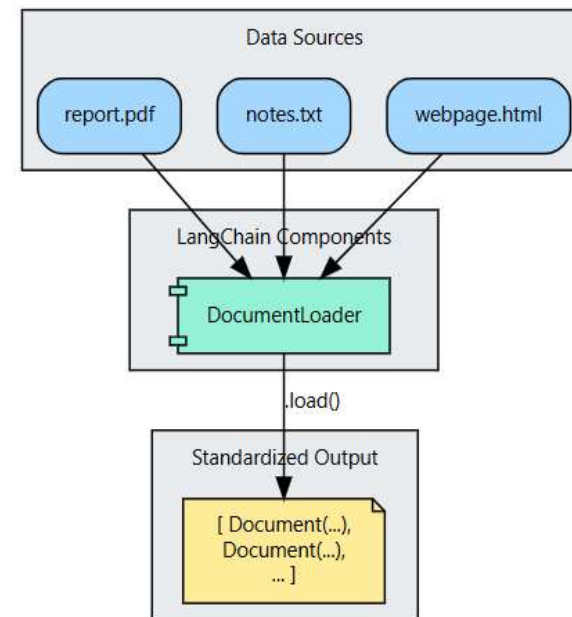
loader = TextLoader("sample.txt")
docs = loader.load()

print(len(docs))
# 1

print(docs[0].page_content)
# This is the first sentence. This is the second.

print(docs[0].metadata)
# {'source': 'sample.txt'}
```

`TextLoader` in Python to load it. The `.load()` method performs the operation and returns a list of `Document` objects. For `TextLoader`, this is typically a list with a single `Document` containing the entire file's content.



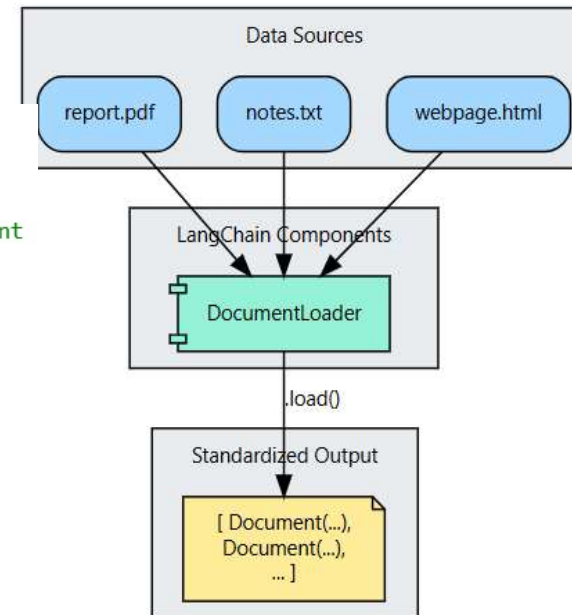
Document Loading

■ Example of DocumentLoader

■ PDF Documents

```
1 # You may need to install the pypdf library: pip install pypdf
2 from langchain_community.document_loaders import PyPDFLoader
3
4 loader = PyPDFLoader("CN7050 Coursework 25-26.pdf") #use any pdf document
5 pdfdoc = loader.load()
6
7 # Assuming the PDF has multiple pages
8 print(f"Loaded {len(pdfdoc)} pages from the PDF.")
9
10 # Inspect the first page's document
11 first_page_doc = pdfdoc[0]
12 print(first_page_doc.page_content[:200]) # Print first 200 characters
13 # ... content of the first page ...
14
15 print(first_page_doc.metadata)
16 # {'source': 'product_manual.pdf', 'page': 0}
```

```
Loaded 5 pages from the PDF.
1 SCHOOL OF ARCHITECTURE, COMPUTING, & ENGINEERING CN7050 Coursework
Guidelines Coursework instructions □ Include a Cover page to the front of
the assignment with your student ID, Name and title
{'source': 'CN7050 Coursework 25-26.pdf', 'page': 0}
```

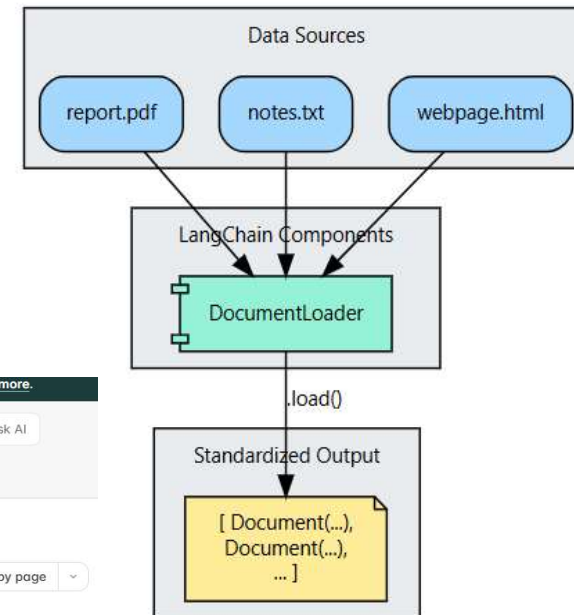
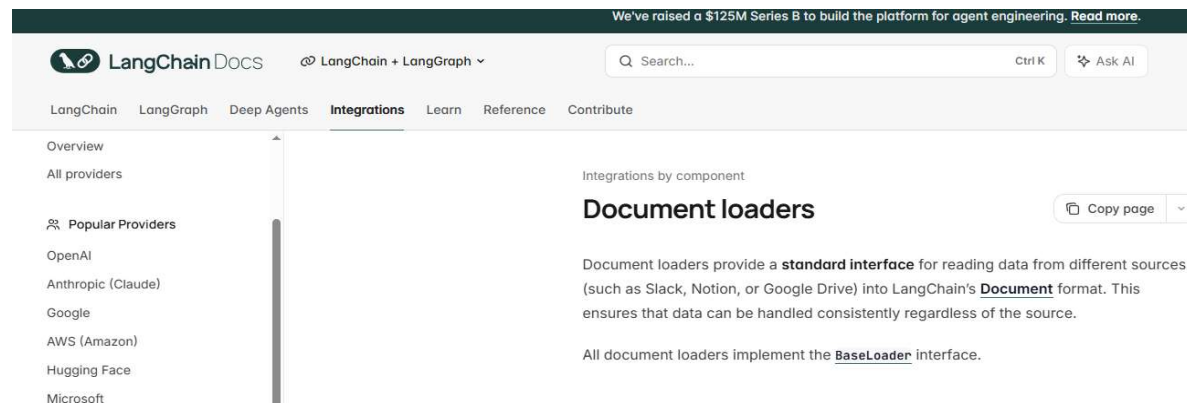


Document Loading

■ Example of DocumentLoader

- Document loaders provide a standard interface for reading data from different sources into LangChain's [Document](#) format.
- see the different sources from here:

https://docs.langchain.com/oss/python/integrations/document_loaders



Splitting Documents for Processing

- Directly feeding a large document to an LLM for question-answering is impractical for two primary reasons.
 - First, it will likely **exceed** the **model's context window** limit.
 - Second, providing a large, unfocused block of text makes it difficult for the model to locate the specific information needed to answer a query.
- The solution is to break down large documents **into smaller, semantically meaningful chunks**. This process, known as **text splitting or chunking**

Splitting Methods

■ Recursive Character Splitting

- The most recommended and versatile method for text splitting in LangChain is the **RecursiveCharacterTextSplitter**
 - this splitter works with a prioritized list of separators.
 - It attempts to split the text using the first separator in the list (by default, `["\n\n", "\n", " ", ""]`).
 - If the resulting chunks are still too large, it moves to the next separator, and so on.
-
- Two important parameters control its behavior:
 - **chunk_size**: Defines the maximum length of each chunk, measured in characters.
 - **chunk_overlap**: Specifies the number of characters to overlap between adjacent chunks. This is a powerful feature that helps maintain continuity of context. If a sentence is cut off at the end of one chunk, the overlap ensures it is completed in the next, reducing the chance of losing relationships between sentences.

Splitting Methods

- While **RecursiveCharacterTextSplitter** is a strong default, LangChain offers other splitters tailored for different needs:
 - **CharacterTextSplitter**: The simplest splitter. It splits text based on a single separator character, such as `\n`. It is fast but less effective at preserving semantic boundaries compared to the recursive approach.
 - **TokenTextSplitter**: This splitter divides text based on a specific number of tokens rather than characters. This is a more precise method for ensuring chunks do not exceed an LLM's context limit, but it requires an external tokenizer.

Splitting Method: Example

RecursiveCharacterTextSplitter

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
```

```
# Some long text loaded from a document
```

```
long_text = """LangChain provides a comprehensive framework for building LLM applications. It simplifies the process of chaining models, managing prompts, and connecting to data sources.
```

```
One of its main components is the TextSplitter. This utility is essential for Retrieval Augmented Generation (RAG), as it breaks down large documents into manageable chunks. These chunks are then embedded and stored in a vector database for efficient retrieval. The choice of chunk size and overlap is significant for application performance."""
```

```
# Initialize the splitter
```

```
text_splitter = RecursiveCharacterTextSplitter(  
    chunk_size=150,  
    chunk_overlap=20  
)
```

```
# Split the document
```

```
chunks = text_splitter.split_text(long_text)
```

```
# Print the resulting chunks
```

```
for i, chunk in enumerate(chunks):  
    print(f"--- Chunk {i+1} ---\n{chunk}\n")
```

```
--- Chunk 1 ---
```

```
LangChain provides a comprehensive framework for building LLM applications. It simplifies the process of chaining models, managing prompts, and connecting
```

```
--- Chunk 2 ---
```

```
connecting to data sources.
```

```
One of its main components is the TextSplitter. This utility is essential for Retrieval Augmented Generation (RAG), as it
```

```
--- Chunk 3 ---
```

```
it breaks down large documents into manageable chunks. These chunks are then embedded and stored in a vector database for efficient retrieval. The choice
```

```
--- Chunk 4 ---
```

```
choice of chunk size and overlap is significant for application performance.
```

The overlap is clearly visible; "connecting" appears at the end of the first chunk and the start of the second, ensuring the full phrase is available for context.

Embedding Creation

- Convert the prepared data into **vector representations** (embeddings) that capture semantic meanings.
- RAG systems' performance heavily depends on the quality of embedding models, as they directly influence:
 - the relevance of retrieved information and
 - the accuracy of generated responses.

```
1 from langchain.embeddings.huggingface import HuggingFaceEmbeddings
2
3 # Embedding Model for converting text to numerical representations
4 embedding_model = HuggingFaceEmbeddings(
5     model_name='BAAI/bge-small-en-v1.5'
6 )
```

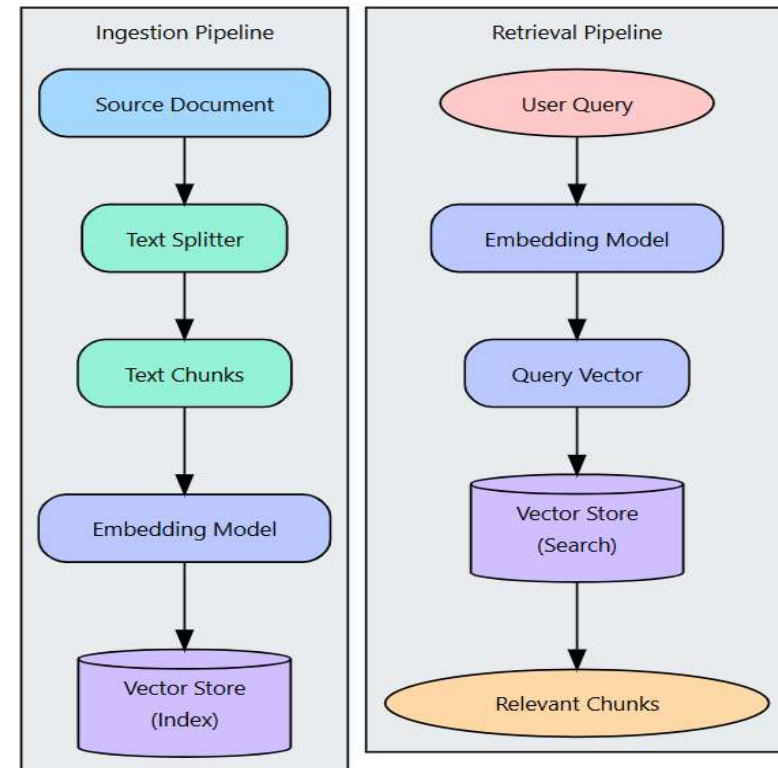
See LangChain Embedding models

https://docs.langchain.com/oss/python/integrations/text_embedding/index#embedding-models

Provider	Model	Embedding Dimensions	Max Tokens
Mistral	mistral-embed	1024	8,000
Google (Vertex AI API)	gemini-embedding-001	3072 (default), 1536, and 768	2,048
Snowflake	snowflake-arctic-embed-l-v2.0	1024	8,192
Voyage AI	voyage-3.5-lite	1024 (default), 256, 512 and 2048	32,000
Voyage AI	voyage-3-large	1024 (default), 256, 512 and 2048	32,000
Google (Vertex AI API)	text-embedding-005	768 (default), 256 and 512	2,048
OpenAI	text-embedding-3-large	3072 (default), 1536 and 1024	8,191
Cohere	embed-v4.0	1536 (default), 256, 512, 1024 and 1536	128,000
OpenAI	text-embedding-3-small	1536 (default), 512 and 1024	8,191
Amazon Bedrock	amazon.titan-embed-text-v2:0	1024 (default), 512 and 256	8,192
Cohere	embed-english-light-v3.0	384	512

Vector Store

- A vector store is a **specialized database** optimized for one primary task: performing extremely fast similarity searches on large collections of vectors.
- It accomplishes this using algorithms for **Approximate Nearest Neighbor (ANN)** search, which find the "closest" vectors without needing to compare the query to every single entry.
- A vector store performs two main jobs in a RAG pipeline:
 1. **Storage and Indexing:** It stores document chunks and their corresponding embedding vectors. During this process, it builds an efficient index to enable fast searching.
 2. **Search and Retrieval:** When given a query vector, it uses its index to quickly retrieve the k^k most similar vectors, returning the document chunks associated with them.



Creating Vector Store

- LangChain includes a suite of integrations with different vector store technologies. Such as:
- **Chroma**, **FAISS**, Milvus, MongoDB, PGVector, PGVectorStore, Pinecone, Qdrant

Vector Store Example: Using FAISS

```
from langchain.vectorstores import FAISS

# Create a local vector database
VectorStore = FAISS.from_texts(texts, embedding_model)
```

Vector Store Example: Using Chroma

```
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings

# Initialize the embeddings model

embeddings = OpenAIEmbeddings()

vectorstore = Chroma.from_documents(documents, embeddings)
```

Select vector store:

- In-memory
- Astra DB
- Azure Cosmos DB NoSQL
- Azure Cosmos DB Mongo vCore
- Chroma
- FAISS
- Milvus
- MongoDB
- PGVector
- PGVectorStore
- Pinecone
- Qdrant

Fetching Data with Retrievers

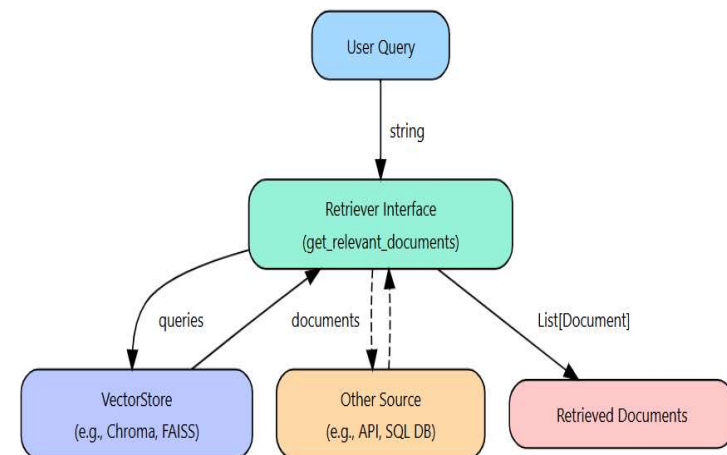
- LangChain provides **Retriever** interface for querying vector store for finding information relevant to a user's question.
- The **retriever** acts as a *generic interface between the application's logic and various underlying data sources*

The most common way to instantiate a retriever is directly from an existing vector store object using `as_retriever()` method

```
# Assuming 'vectorstore' is an initialized Chroma or FAISS vector store object
retriever = vectorstore.as_retriever()

# The retriever is now ready to be used
query = "What is the architecture of a RAG system?"
relevant_docs = retriever.get_relevant_documents(query)

# Print the content of the first retrieved document
print(relevant_docs[0].page_content)
```



Building Prompt (Augmentation Process)

```
from langchain import PromptTemplate

template = """<|user|>
You are an assistant that extracts specific information from
academic PDF documents. Use the following pieces of context to
answer the question. If you don't know the answer, say that you
don't know.

Context from document:
{context}

Question: {question}

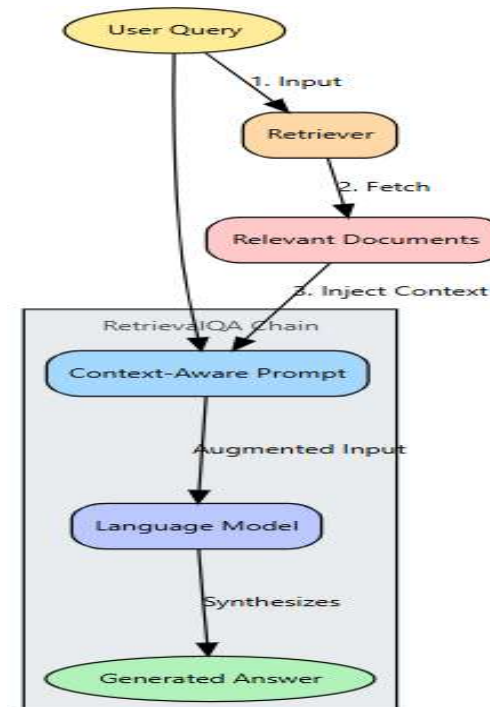
Instruction:
- Provide only the direct answer.
- Do not include explanations, context, or repeated information.
- If the answer is a date, return only the date in this format:
YYYY-MM-DD or the exact format found in the document.
<|end|>
<|assistant|>"""

prompt = PromptTemplate(
    template=template,
    input_variables=["context", "question"]
)
```

- A prompt acts as the **instruction set** between retrieved context (from the PDF or database) and the LLM:
- Even though the LLM is powerful, it does **not automatically know what to do** with the retrieved documents.
- You must tell it *how* to use the retrieved information and *what format* you expect in the answer.
- **{context}** and **{question}** are placeholder variables.
 - **{context}** is replaced by retrieved context
 - **{question}** is replaced with user actual user's query

Building RAG System

- Building RAG utilizes several components:
 - **Retriever** designed to fetch relevant documents from a vector store
 - **LLM** capable of generating text.
 - **Prompt** context aware prompt which guides how the LLM should respond
- Combining these elements creates a cohesive application, allowing it to take a user's question and produce a well-grounded answer.



Building RAG Pipeline

RetrievalQA (LangChain Class) → Builds a **RAG** pipeline by combining:

- **Retriever** → Fetches relevant information from your document database
- **LLM** → Generates the final answer using the retrieved information
- **Prompt** → Guides how the LLM should respond

Key Components:

- **from_chain_type** → Defines
 - Which **LLM** to use
 - How to combine retrieved chunks (**chain_type**)
 - Which **prompt** template to apply
- **retriever** → The configured retriever object

```
1 #LLM
2 from langchain import LlamaCpp
3
4 # Make sure the model path is correct for your system!
5 llm = LlamaCpp(
6     model_path="Phi-3-mini-4k-instruct-q4.gguf",
7     n_gpu_layers=-1,
8     max_tokens=500,
9     n_ctx=2048,
10    seed=42,
11    verbose=False
12 )
```

```
1 # RAG Pipeline
2 rag = RetrievalQA.from_chain_type(
3     llm=llm,
4     chain_type='stuff',
5     retriever=db.as_retriever(),
6     chain_type_kwargs={
7         "prompt": prompt
8     },
9     verbose=True
10 )
```

RAG Applications

Healthcare: Agentic RAG systems analyze patient data — such as medical history and test results — to generate personalized treatment plans, improving diagnostics and care.

Finance: In finance, these systems adapt to market changes in real-time, retrieving current financial data to provide insights that inform trading strategies and risk management.

Research and Data Analysis: Agentic RAG holds immense potential for applications such as research, data analysis, and knowledge exploration.

Customer Support: In customer support scenarios, Agentic RAG can provide more nuanced and accurate responses by considering multiple information sources and previous interaction contexts.

Legal and Compliance: In legal and compliance sectors, Agentic RAG assists professionals in navigating and explaining intricate documentation while maintaining accuracy and relevance.

Education: Agentic RAG systems can serve as intelligent tutors, providing personalized learning experiences by adapting to individual student needs and retrieving relevant educational content.

Human Resources: In HR, these systems can streamline recruitment by analyzing candidate data, matching qualifications with job requirements, and assisting in decision-making processes.

E-commerce: Agentic RAG can enhance product recommendations by analyzing customer preferences and behaviors, leading to improved customer satisfaction and sales.

Summary

- RAG architecture and its component
 - Retriever
 - Generation
- Steps to construct RAG System
 - Loading and Splitting the Document
 - Creating Embeddings and the Vector Store
 - Building the RetrievalQA Chain
 - Asking Questions

Next: LLM-powered Autonomous Agent System

